

MATLAB TUTORIAL

Second Laboratory Report for CENG 3264

Submitted by
Aloniab Negash
2135277

Computer Engineering
University of Houston - Clear Lake
Houston, Texas 77058

9/8/2026

ABSTRACT

This report documents the completion of the second CENG 3264 laboratory, which introduces the MATLAB software environment through three tasks. Task 1 covered the generation of sinusoidal, exponential, unit impulse, unit step, and random signals. Task 2 covered basic signal manipulation operations, including delay, addition, and multiplication, applied to three sinusoidal signals of different frequencies. Task 3 involved writing a new M-file to load an external speech recording (watchdog.wav), generate Gaussian noise, scale that noise to a target signal-to-noise ratio, and mix it with the speech signal to observe the effect of additive noise on audio intelligibility. Through these exercises, familiarity was gained with the MATLAB interface, scripting, plotting, and fundamental digital signal processing concepts.

1. INTRODUCTION

This report documents the completion of Lab 2 for CENG 3264, which introduces the MATLAB software environment. The lab consists of three tasks: (1) generation of standard test signals (sinusoidal, exponential, impulse, step, and random signals), (2) basic signal manipulation operations (delay, addition, and multiplication), and (3) creation of an M-file to load an external .wav audio file, contaminate it with Gaussian noise at a specified signal-to-noise ratio (SNR), and analyze the result. The purpose of these exercises is to build familiarity with the MATLAB interface, scripting, plotting, and basic digital signal processing concepts.

2. TASK 1: SIGNAL GENERATION

Task one of the labs focused on generating different signals. We were given different types of signals along with their code. We were able to add a new function to the script in MATLAB, and that enabled us to generate a new signal wave and a new graph. Below are the new signal graphs and code used to generate these signals:

2.1 Sinusoidal Signal Generation

Part (a): 64-sample cosine wave, $f = 25$ Hz, $f_s = 400$ Hz, $A = 1.5$ V, phase = 0.

```
% Generating 64 samples of x(t)=sin(2*pi*f*t) with a
% Frequency of 1KHz, and sampling frequency of 8KHz.
N=64; % Define Number of samples
n=0:N-1; % Define vector n=0,1,2,3,...62,63
f=1000; % Define the frequency
fs=400; % Define the sampling frequency
x=1.5*cos(2*pi*(25/400)*n+0); % Generate x(t)
plot(n,x); % Plot x(t) vs. t
grid;
title('Sinewave [f=1KHz, fs=8KHz]');
xlabel('Sample Number');
ylabel('Amplitude');
```

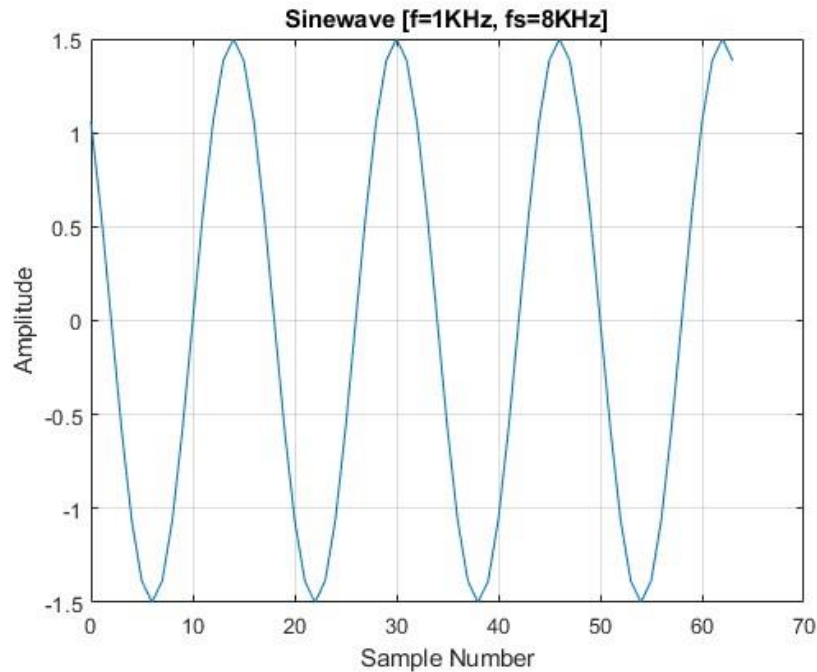


Figure 1. 64-sample cosine wave, $f = 25 \text{ Hz}$, $f_s = 400 \text{ Hz}$, $A = 1.5 \text{ V}$, $\text{phase} = 0$.

Part (b): same signal as (a) but with a phase angle of $\pi/2$.

```
% Generating 64 samples of x(t)=sin(2*pi*f*t) with a
% Frequency of 1KHz, and sampling frequency of 8KHz.
N=64; % Define Number of samples
n=0:N-1; % Define vector n=0,1,2,3,...62,63
f=1000; % Define the frequency
fs=400; % Define the sampling frequency
x=1.5*cos(2*pi*(25/400)*n+pi/2); % Generate x(t)
plot(n,x); % Plot x(t) vs. t
grid;
title('Sinewave [f=1KHz, fs=8KHz]');
xlabel('Sample Number');
ylabel('Amplitude');
```

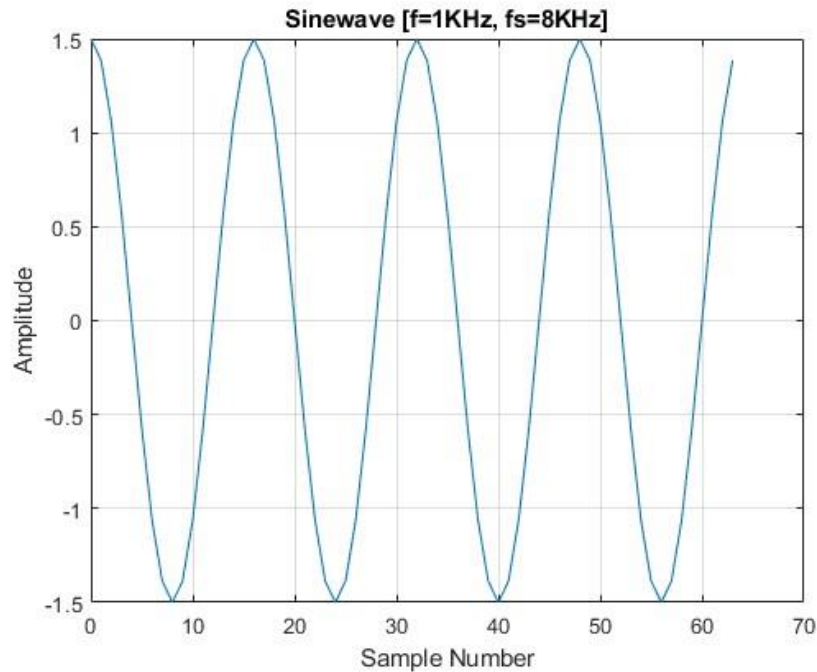


Figure 2. 64-sample cosine wave, identical to Figure 1 but shifted by a $\pi/2$ phase angle.

Comparing Figures 1 and 2, the added phase term shifts the waveform horizontally along the sample axis without changing its frequency or amplitude, confirming the expected effect of the phase parameter θ in the cosine expression.

2.2 Exponential Signal Generation

Signal generated: a decaying sinusoid combining an exponential term with a sine term over $t = 0$ to 40 mS.

```
% Generating the signal x(t)=exp(-0.1t)
t=0:0.1:40;
x=exp(-0.1*t)+ sin(0.6*t);
plot(t,x);
grid;
title('Exponential Signal');
xlabel('Time [mS]');
ylabel('Amplitude');
```

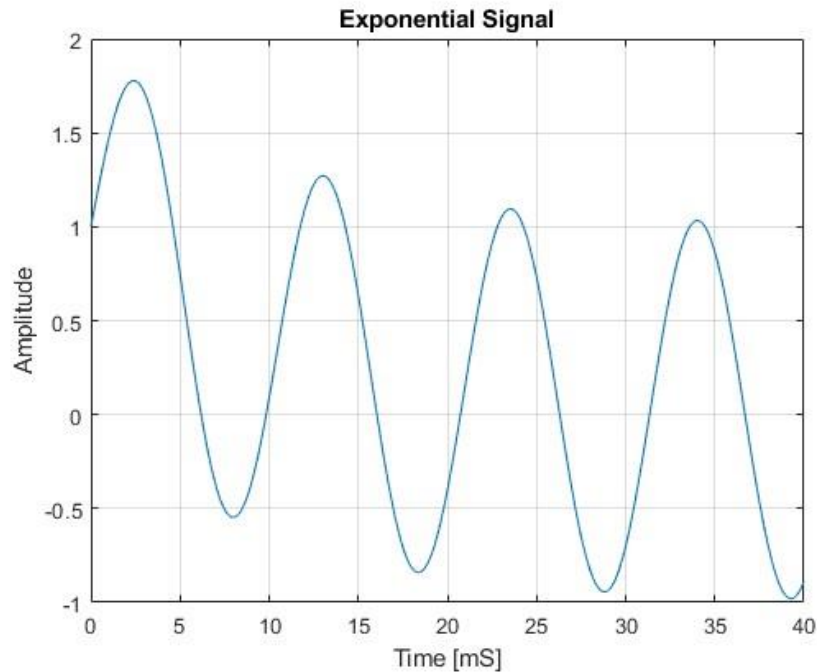


Figure 3. Exponential/sinusoidal combination signal for $t = 0$ to 40 ms.

The resulting waveform oscillates while its overall amplitude envelope decreases over time, illustrating how the decaying exponential term influences the sinusoidal component as t increases.

2.3 Unit Impulse Signal Generation

46 samples ($n = -23$ to 22) were generated for two impulse combinations:

$$(a) x(n) = 2\delta(n - 10) \quad (b) x(n) = 5\delta(n - 10) + 2.5\delta(n - 20)$$

```
% Generating 46 Samples of a unit impulse signal
N= 46; % Define the number of samples
n=-(N/2):((N/2)-1); % n = -23:22
x=zeros(1,N);

% part a
xa=zeros(1,N);
xa(n==10) = 2;

% part b
xb= zeros(1,N);
xb(n== 10) =5;
xb(n ==20) = 2.5;

% plot
figure;
```

```

subplot(2,1,1);
stem(n,xa, 'filled'); % Plot the impulse
grid; title('x_a(n) = 2\delta(n-10)');
subplot(2,1,2);
stem(n, xb,'filled');
grid; title('x_b(n) = 5\delta(n-10) + 2.5\delta(n-20)');
title('A Unit Impulse Signal');
xlabel('Sample Number');
ylabel('Amplitude');

```

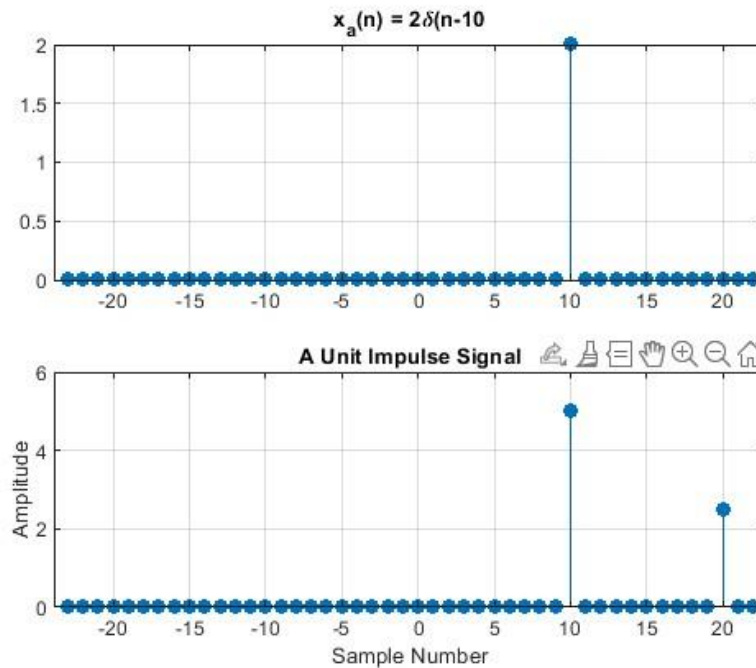


Figure 4. Unit impulse signals: (top) $x_a(n) = 2\delta(n-10)$; (bottom) $x_b(n) = 5\delta(n-10) + 2.5\delta(n-20)$.

The top plot shows a single impulse of height 2 located at $n = 10$. The bottom plot shows two impulses — height 5 at $n = 10$ and height 2.5 at $n = 20$ — confirming that each delta term in the expression places an independent, correctly-scaled spike at its respective sample location.

2.4 Unit Step Signal Generation

40 samples were generated for two step-based signals using an appropriate discrete time scale ($n = -20:20$):

(a) $x(n) = u(n) - u(n-1)$

```

% Generates 40 samples of a unit step signal, u(n)
N=40; % Define the number of samples
n=-20:20; % Define a suitable discrete time axis
u=zeros(1,(N/2)+1),ones(1,(N/2)); % Generate the signal

```

```

plot(n,u); % Plot the signal
x(n)= u(n)- u(n-1);
axis([-20,+20,-0.5,1.5]); % Scale axis
grid;
title('A Unit Step Signal');
xlabel('Sample Number');
ylabel('Amplitude');

```

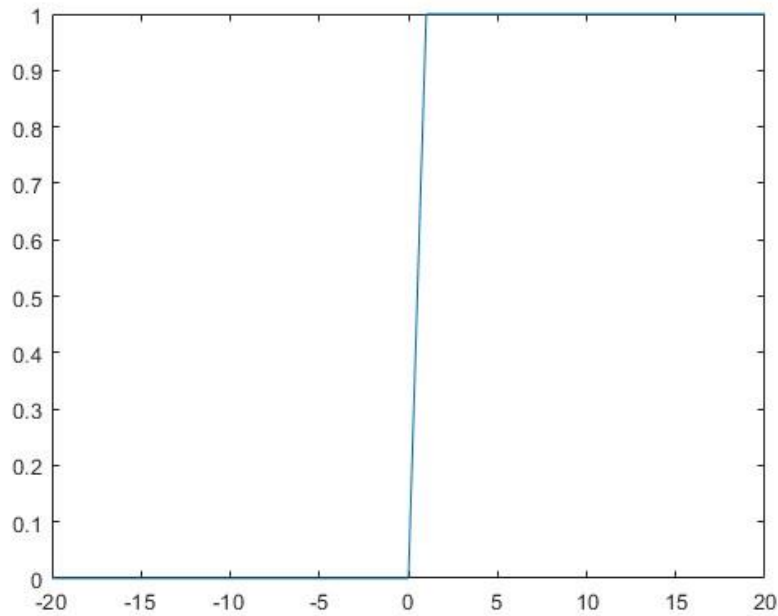


Figure 5. Unit step signal used to construct $x(n) = u(n) - u(n - 1)$.

(b) $g(n) = u(n - 1) - u(n - 5)$

```

% Generates 40 samples of a unit step signal, u(n)
N=40; % Define the number of samples
n=-20:20; % Define a suitable discrete time axis
u=[zeros(1,(N/2)+1),ones(1,(N/2))]; % Generate the signal
plot(n,u); % Plot the signal
g(n)=u(n-1)-u(n-5);
axis([-20,+20,-0.5,1.5]); % Scale axis
grid;
title('A Unit Step Signal');
xlabel('Sample Number');
ylabel('Amplitude');

```

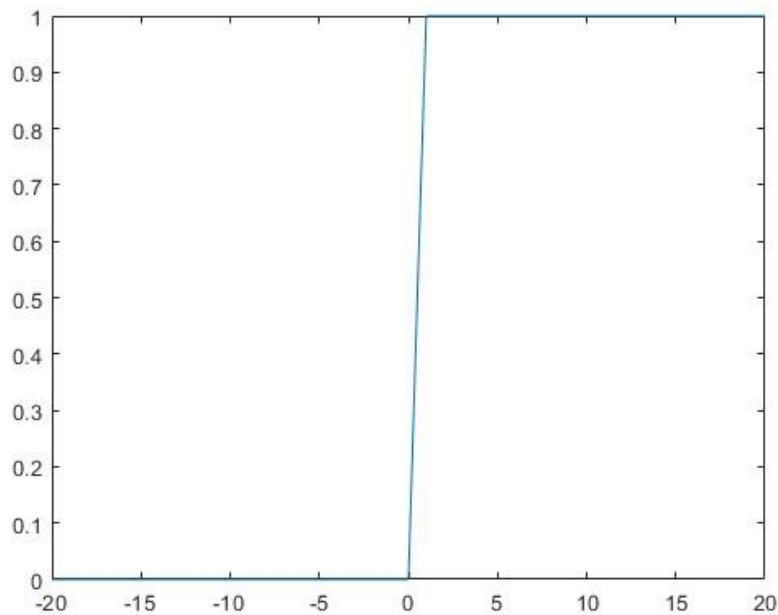


Figure 6. Unit step signal used to construct $g(n) = u(n - 1) - u(n - 5)$.

Both plots show the base unit step function $u(n)$ transitioning from 0 to 1 at the origin, which when combined via subtraction (as coded) produces the pulse-shaped signals $x(n)$ and $g(n)$ described in the question.

2.5 Generating Random Signals

Normal (Gaussian) and uniformly distributed random signals, 128 samples each, were generated and plotted alongside their histograms.

```
% Generates Uniformly and Normally Distributed random signals
N=128; % Define Number of samples
R1=randn(1,N); % Generate Normal Random Numbers
R2=rand(-3.14,N); % Generate Uniformly Random Numbers
figure(1); % Select the figure
subplot(2,2,1); % Subdivide the figure into 4 quadrants
plot(R1); % Plot R1 in the first quadrant
grid;
title('Normal [Gaussian] Distributed Random Signal');
xlabel('Sample Number'); ylabel('Amplitude');
subplot(2,2,2); % Select the second quadrant
hist(R1); % Plot the histogram of R1
grid;
title('Histogram [Pdf] of a normal Random Signal');
xlabel('Sample Number'); ylabel('Total');
subplot(2,2,3);
```

```

plot(R2);
grid;
title('Uniformly Distributed Random Signal');
xlabel('Sample Number'); ylabel('Amplitude');
subplot(2,2,4);
hist(R2);
grid;
title('Histogram [Pdf] of a uniformly Random Signal');
xlabel('Sample Number'); ylabel('Total');

```

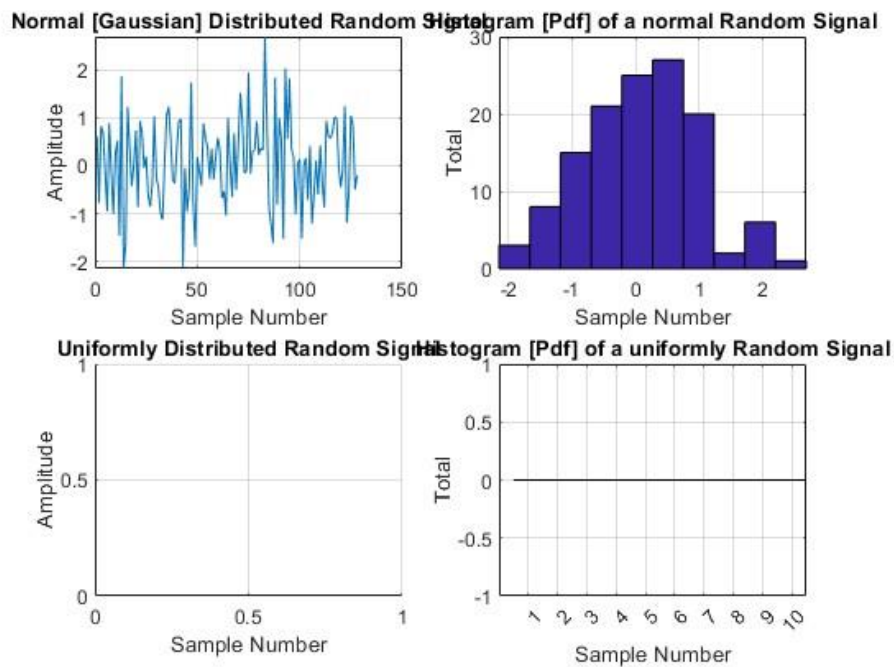


Figure 7. Random signal generation: (top) normal/Gaussian distributed signal and histogram; (bottom) uniformly distributed signal and histogram.

The top two plots show the normally distributed signal R1 and its bell-shaped histogram, consistent with a Gaussian distribution centered near zero. The bottom two plots, generated with $R2 = \text{rand}(-3.14, N)$, came out empty — $\text{rand}()$ only accepts a dimension argument, not a lower bound, so this line did not generate the intended uniform values between $-\pi$ and π . Generating 128 uniform random numbers over that specific range instead requires scaling the standard $\text{rand}()$ output, for example: $R2 = -\pi + (2*\pi)*\text{rand}(1, N)$.

3. TASK 2: SIGNAL MANIPULATION

For this task in this Lab, we are working with a MATLAB code where we will be doing signal manipulation, this includes various types of manipulation: Signal Shifting/Delay, Signal Addition/Subtraction, Signal Amplification/Attenuation, Signal Differentiation/Integration, and Signal Multiplication/Division. The codes we use will cover all of these signal manipulations and the program will display figures on a new tab and label them.

Our goal is to use MATLAB to visualize these signal manipulations on graphs based on the diagrams provided on the Task guidelines, this also helps us get familiarized with the program and its features to possibly use it in a future project requiring coding and graphing. This task resulted in a total of 6 graphs, each graph visualizing each type of manipulation, so we do get to see the differences of each type of manipulation using coding, and we concluded that they were very different from each other from the graphs alone.

.

3.1 Basic Signal Generation (x1, x2, x3)

In this code we replicate a basic signal manipulation, we code in the values of the 3 different grids and title them (x1, x2,x3) and plot the points on the grid. The values for the grids are all different equations based on sine.

```
% Program demonstrating Basic Signal Manipulation
N=128;
f1=150;
f2=450;
f3=1500;
fs=8000;
n=0:N-1;
x1=sin(2*pi*(f1/fs)*n);
x2=(1/3)*sin(2*pi*(f2/fs)*n);
x3=sin(2*pi*(f3/fs)*n);
figure(1);
subplot(1,1,1);
subplot(2,3,1);
plot(n,x1);
```

```

grid;
title('Signal, x1(n)');
subplot(2,3,2);
plot(n,x2);
grid;
title('Signal, x2(n)');
subplot(2,3,3);
plot(n,x3);
grid;
title('Signal, x3(n)');

```

3.2 Signal Delay

For the second graph, we replicate a signal delay manipulation, we only have 1 graph displayed, called $x(n)$ here we plot the values based on n , and the original graph $x1$. Plugging in these values into the grid in order to project the graph.

```

% Signal Delay
x1d=[zeros(1,20), x1(1:N-20)];
subplot(2,3,4);
plot(n,x1d);
grid;
title('Delayed x(n), [x1(n-20)]');

```

3.3 Signal Addition

For the signal addition, we also only display one graph, were we do an equation were we do the addition of graph $x1$ and $x2$, and after plotting n and the addition of $x1$ and $x2$, we get the final graph.

```

% Signal Addition
xadd=x1+x2;
subplot(2,3,5);
plot(n,xadd);
grid;
title('x1(n)+x2(n)');

```

3.4 Signal Multiplication

For this last graph we are doing signal multiplication manipulation, were the graph $x1$ and $x3$ are multiplied by each other, and on this last graph we just plot the $xmult$ graph with its values instead of also doing n .

```
% Signal Multiplication
xmult=x1.*x3;
subplot(2,3,6);
plot(xmult);
grid;
title('x1*x3');
```

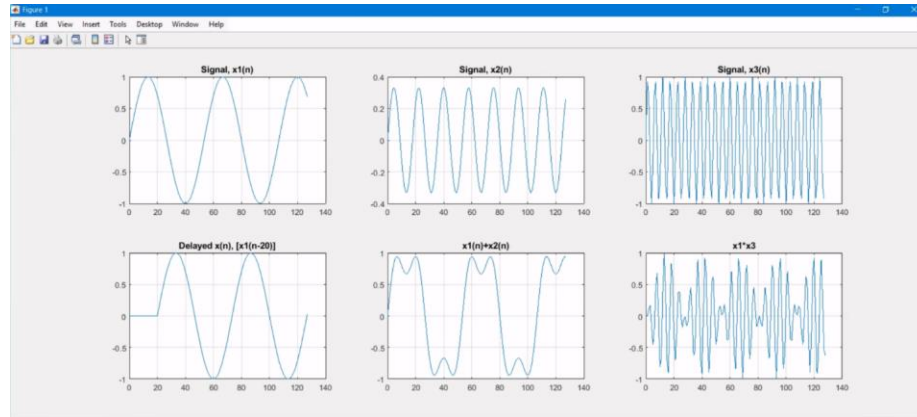


Figure 8. Combined results of Task 2: original signals $x1$, $x2$, $x3$ (top row); delayed $x1$, sum $x1+x2$, and product $x1 \times x3$ (bottom row).

Each manipulation produces a visually distinct result: the delayed signal is $x1$ shifted right with leading zeros; the addition combines the shapes of the two lower-frequency signals into a single waveform; and the multiplication of $x1$ with the much higher-frequency $x3$ produces a rapidly-varying, amplitude-modulated pattern. This confirmed that the original signals $x1$, $x2$, and $x3$ must be generated and plotted first, since the delay, addition, and multiplication operations all depend directly on their values — attempting the later operations without the base signals defined results in a program error.

4. TASK 3: GENERATE M-FILES AND LOAD EXTERNAL FILES

This task loads an external speech recording (watchdog.wav), generates Gaussian noise, scales the noise to achieve a target 10 dB signal-to-noise ratio (SNR), and mixes it with the speech signal to simulate a noisy communication channel.

4.1 Loading and Playing the Original Speech Signal

The watchdog.wav file was loaded using wavread, which returns the sample data (y) and the sampling frequency (fs). The signal was plotted and played back using sound(y,fs) to confirm it loaded correctly.

```
clear all; close all; clc;
[y, fs] = wavread('watchdog.wav');
n = length(y);

figure
plot(y)
title('Original Speech Signal')
xlabel('Sample Number'); ylabel('Amplitude')
sound(y, fs)
pause(length(y)/fs + 1)
```

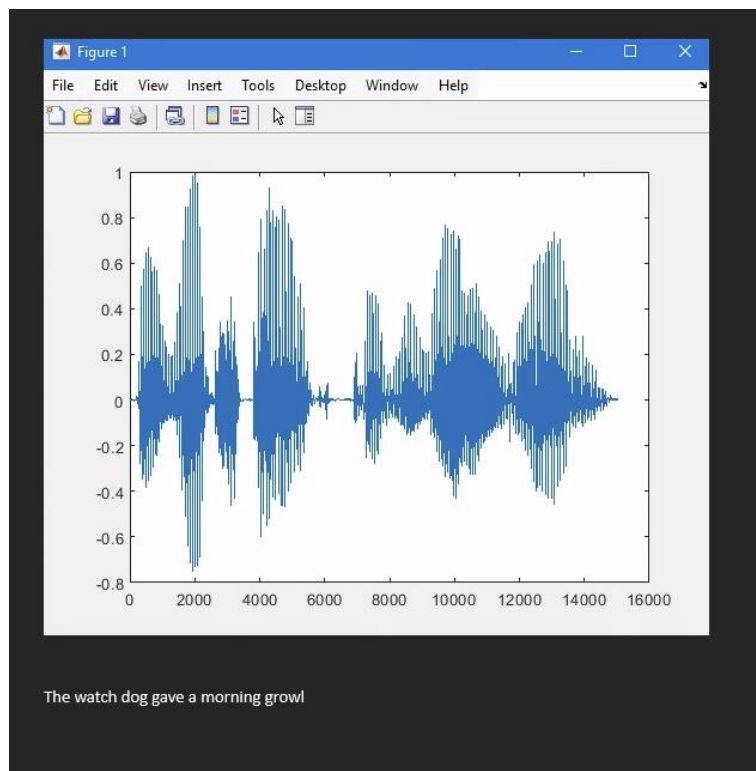


Figure 9. Waveform of the original watchdog.wav speech signal (amplitude vs. sample number, ~15,000 samples).

Figure 9 shows several distinct high-amplitude bursts separated by near-silent gaps, each corresponding to a spoken word or syllable in the recording. Playing the file back confirmed the spoken phrase: “The watch dog gave a morning growl.” The number of visible amplitude bursts in the waveform (approximately six to seven) lines up with the number of words/syllable groups

in that sentence, confirming the file was read correctly and the sample data accurately represents the audio.

4.2 Results: What Was Heard

Three separate audio playbacks occurred during execution of the script, each pausing until playback finished:

1. Original speech (`sound(y,fs)`): the clean recording played clearly and was fully intelligible — “The watch dog gave a morning growl.”
2. Scaled noise alone (`sound(noise,fs)`): played as a hiss/static-like sound with no discernible speech content, as expected for Gaussian noise.
3. Mixed signal (`sound(s,fs)`): the sentence remained largely understandable despite the added noise, though a noticeable hiss was audible underneath the speech at the 10 dB SNR level — a clearly degraded but still intelligible version of the original recording.

5. CONCLUSIONS

This lab introduced the MATLAB software environment and its interface through three hands-on tasks. For task one of the labs, we were introduced to the MATLAB software. We familiarize ourselves with its Interface and learned how to use basic functions in MATLAB. Through hands-on practice, we generated sinusoidal, exponential, unit impulse, unit step, and random signals by modifying the script with a new function. After inputting the new function into the code and running the program, we were able to observe changes in the signal graph in real time.

For task 2, we have seen how these different manipulations look graphically, and how their appearances are based on a combination of the three original graphs (x_1 , x_2 , x_3) and that results are extremely different based on the type of manipulation being performed. Based on the code, we learned that all points need to be planted on the code, the titles are hardcoded, and how we plot both the main points (n and x) and the subplots, and we code in the grid.

In Summary, aside from learning about the different type of signal manipulations, we also learned how we are able to code them into MATLAB correctly in order to get the most accurate graph possible, and how in order to do things like signal addition and multiplication, we first need to plot and code the original signals, if not, coding the rest of the manipulations will result in a program error.

In Task 3, a new M-file was created to load an external audio recording, generate Gaussian noise, scale it to a target 10 dB SNR, and mix it with the speech signal; the resulting audio remained intelligible despite the added noise, and figures of the waveform, the noise distribution, and their amplitude overlay were produced to document the process. Together, these exercises built practical familiarity with MATLAB scripting, plotting, and basic digital signal processing concepts that will be useful in future coursework and projects.

REFERENCES

- [1] CENG 3264 Course Staff, “Lab 2: MATLAB Tutorial I,” Course Handout, University of Houston-Clear Lake, 2026.
- [2] MathWorks, “MATLAB Documentation,” The MathWorks, Inc. [Online]. Available: <https://www.mathworks.com/help/matlab/>

APPENDIX A: TASK 1 MATLAB CODE

```
%% Sinusoidal Signal Generation
% Part (a)
N=64; n=0:N-1; f=1000; fs=400;
x=1.5*cos(2*pi*(25/400)*n+0);
plot(n,x); grid;
title('Sinewave [f=1KHz, fs=8KHz]');
xlabel('Sample Number'); ylabel('Amplitude');

% Part (b)
x=1.5*cos(2*pi*(25/400)*n+pi/2);
plot(n,x); grid;
title('Sinewave [f=1KHz, fs=8KHz]');
xlabel('Sample Number'); ylabel('Amplitude');

%% Exponential Signal Generation
t=0:0.1:40;
x=exp(-0.1*t)+ sin(0.6*t);
plot(t,x); grid;
title('Exponential Signal');
xlabel('Time [mS]'); ylabel('Amplitude');

%% Unit Impulse Signal Generation
N= 46; n=-(N/2):(N/2)-1;
xa=zeros(1,N); xa(n==10) = 2;
xb=zeros(1,N); xb(n==10)=5; xb(n==20) = 2.5;
figure;
subplot(2,1,1); stem(n,xa,'filled'); grid;
title('x_a(n) = 2\delta(n-10)');
subplot(2,1,2); stem(n,xb,'filled'); grid;
title('x_b(n) = 5\delta(n-10) + 2.5\delta(n-20)');
xlabel('Sample Number'); ylabel('Amplitude');

%% Unit Step Signal Generation
N=40; n=-20:20;
u=[zeros(1,(N/2)+1),ones(1,(N/2))];
plot(n,u); x(n)= u(n)- u(n-1);
axis([-20,+20,-0.5,1.5]); grid;
title('A Unit Step Signal');
xlabel('Sample Number'); ylabel('Amplitude');

plot(n,u); g(n)=u(n-1)-u(n-5);
axis([-20,+20,-0.5,1.5]); grid;
title('A Unit Step Signal');
xlabel('Sample Number'); ylabel('Amplitude');

%% Random Signal Generation
N=128;
R1=randn(1,N);
R2=rand(-3.14,N);
figure(1);
subplot(2,2,1); plot(R1); grid;
title('Normal [Gaussian] Distributed Random Signal');
subplot(2,2,2); hist(R1); grid;
title('Histogram [Pdf] of a normal Random Signal');
subplot(2,2,3); plot(R2); grid;
title('Uniformly Distributed Random Signal');
subplot(2,2,4); hist(R2); grid;
title('Histogram [Pdf] of a uniformly Random Signal');
```


APPENDIX B: TASK 2 MATLAB CODE

```
% Program demonstrating Basic Signal Manipulation
N=128;
f1=150; f2=450; f3=1500; fs=8000;
n=0:N-1;
x1=sin(2*pi*(f1/fs)*n);
x2=(1/3)*sin(2*pi*(f2/fs)*n);
x3=sin(2*pi*(f3/fs)*n);
figure(1);
subplot(2,3,1); plot(n,x1); grid; title('Signal, x1(n)');
subplot(2,3,2); plot(n,x2); grid; title('Signal, x2(n)');
subplot(2,3,3); plot(n,x3); grid; title('Signal, x3(n)');

% Signal Delay
x1d=[zeros(1,20), x1(1:N-20)];
subplot(2,3,4); plot(n,x1d); grid;
title('Delayed x(n), [x1(n-20)]');

% Signal Addition
xadd=x1+x2;
subplot(2,3,5); plot(n,xadd); grid;
title('x1(n)+x2(n)');

% Signal Multiplication
xmuilt=x1.*x3;
subplot(2,3,6); plot(xmuilt); grid;
title('x1*x3');
```

APPENDIX C: TASK 3 MATLAB CODE

```
clear all; close all; clc;

%% Read file
[y, fs] = audioread('watchdog.wav'); % use audioread (wavread is deprecated)
n = length(y);

figure
plot(y)
title('Original Speech Signal')
xlabel('Sample number')
ylabel('Amplitude')
sound(y, fs)
pause(length(y)/fs + 1) % wait for playback to finish before continuing

%% Generate Gaussian noise
noise = randn(n,1); % mean = 0, std = 1
figure
subplot(2,1,1)
plot(noise)
title('Gaussian Noise Waveform')

subplot(2,1,2)
hist(noise, 100)
title('Histogram of Gaussian Noise')

%% Add noise to speech at target SNR
dB = 10;
sf = 10.^(dB/20);
rmsy = rms(y);
rmsn = rms(noise);
noise = noise .* (rmsy./rmsn) ./ sf;

sound(noise, fs) % listen to the pure scaled noise
pause(length(noise)/fs + 1)

figure
plot(noise, 'r'); hold on
plot(y, 'b')
axis off
title('Noise (red) vs Speech (blue) Amplitude Overlay')

%% Mix speech + noise
s = y + noise;
sound(s, fs)
pause(length(s)/fs + 1)

%% Verify actual SNR
rmsn = rms(noise);
snr = 20*log10(rmsy./rmsn);
disp(snr)
```